

R1.05 - Introduction aux Bases de Données et SQL

Arnaud Delhay-Lorrain (arnaud.delhay@univ-rennes1.fr)

Département Informatique
IUT de Lannion
Université de Rennes 1

BUT Informatique - Semestre 1

Présentation du cours

- Ressource R1.05, complété par la SAÉ¹ S1.04 création d'une base de données
- Forme de l'enseignement :
 - ▶ Période P2 (5 semaines) :
 - ★ Cours Magistral (1h/semaine) + 1 CM initial
 - ★ Travaux Dirigés (2h/semaine)
 - ★ Travaux Pratiques (2h/semaine)
 - ▶ Période P3 (6 semaines) :
 - ★ Travaux Dirigés (1h/semaine dont 2H en FC²)
 - ★ Travaux Pratiques (2h/semaine dont 2h en FC)
 - ★ SAÉ : Projet (1h encadrée + 1h en autonomie /semaine)

-
1. Situation d'Apprentissage et d'Évaluation
 2. Formation Complémentaire pour la SAÉ

Présentation du cours

Évaluation

- Devoir surveillé (1h) : en fin de S1, semaine du 17 janvier 2022
- Contrôles Travaux Pratiques (sur table 1h) : en continu, selon les groupes
- Calcul de la moyenne
 - ▶ Moyenne R1.05 = $(2 \times \text{note DS} + \text{Moyenne TP})/3$

Plan du cours

- 1 Motivations
- 2 Modélisation d'un SI - notions élémentaires
- 3 Les bases du Modèle Relationnel (MR)
- 4 Opérateurs et Langage de Manipulation de Données (LMD)
- 5 Passage à PostgreSQL

Motivations

1 Motivations

- Qu'est-ce qu'une BDD ?
- Utilité d'une BDD

Motivations

1 Motivations

- Qu'est-ce qu'une BDD ?
- Utilité d'une BDD

Qu'est-ce qu'une BDD ?

Définitions

Définition

Base de Données (BDD) : Un ensemble organisé d'informations avec un objectif commun.

- Peu importe le support
- Structuration et organisation
 - ▶ stockage en grande quantité
 - ▶ ==> faciliter l'exploitation

Qu'est-ce qu'une BDD ?

Définitions

Définition

Base de Données informatisée : Un ensemble structuré de données enregistrées sur des supports accessibles par l'ordinateur, représentant des informations du monde réel et pouvant être interrogées et mises à jour par une communauté d'utilisateurs.

- Résultat de la conception d'une BDD informatisée : description des données
- Description réalisée avec un modèle de données
- Gestion et accès : Système de Gestion de Bases de Données

Qu'est-ce qu'une BDD ?

Quelques exemples de bases de données ?

- Gestion des inscriptions universitaires
- Gestion des notes
- Site web marchand
- Gestion des inscriptions et déroulement d'une compétition sportive
- Gestion de données personnelles (agenda, répertoire, bibliothèque, *etc.*)
- Album photos

Qu'est-ce qu'une BDD ?

Définitions

Définition

Système de Gestion de Bases de Données (SGBD) : Un ensemble de programmes assurant la gestion et l'accès à une base de données.

Un SGBD est caractérisé par :

- le modèle de description des données qu'il supporte (hiérarchique, réseau, relationnel, objet, ...)
- la description réalisée grâce à un langage de description des données (LDD)

Définition

Schéma : Description suivant un modèle, grâce à un langage de description de données.

Qu'est-ce qu'une BDD ?

Définitions

Les manipulations de données sont faites grâce à :

- un Langage de Manipulation de Données (LMD) spécifique
- ou un langage de programmation classique

Trois catégories d'utilisateurs des bases de données :

- les utilisateurs interactifs (utilisateur quelconque)
- les programmeurs d'applications (ex : développeur web)
- les administrateurs (gestion)

Motivations

1 Motivations

- Qu'est-ce qu'une BDD ?
- Utilité d'une BDD

Dès l'origine, les informaticiens ont observé :

- taille de la RAM insuffisante
- volumes de données en croissance constante (ex : WEB de nos jours)

Les besoins sont donc de :

- gérer des dizaines Tera-octets de données

Enjeux

Défauts à éviter

- Redondance des données
- Incohérence en modification
- Anomalie d'insertion
- Anomalie de suppression

- Gérer des centaines de Tera-octets de données
- Données géographiquement distribuées à l'échelle d'Internet
- Plusieurs dizaines de milliers d'utilisateurs
- Un contexte d'exploitation changeant (débits de communication)
- Voir sur des noeuds volatiles

Les outils

Modèle

Il existe plusieurs modèles de bases de données :

- Modèle hiérarchique
- Modèle réseau
- Modèle relationnel
- Modèle objet

La notion de bases de données objet ou relationnel-objet est plus récente et encore en phase de recherche et de développement. Elle sera très probablement ajoutée au modèle relationnel.

Les outils

Langage

Le langage utilisé pour effectuer des traitements sur les données est le **SQL** (Structured Query Language).

Le SQL est le fruit des recherches sur le modèle relationnel et est normalisé.

Le SQL est un langage déclaratif, en opposition aux langages impératifs (C, Pascal, PHP, *etc.*)

Logiciels : le SGBD

Les qualités d'un bon SGBD sont :

- Indépendance physique
- Indépendance logique
- Accès aux données
- Administration centralisée des données (intégration)
- Non redondance des données
- Cohérence des données
- Partage des données
- Sécurité des données
- Résistance aux pannes

Quelques SGBD connus et utilisés

Il existe de nombreux systèmes de gestion de bases de données, en voici une liste non exhaustive :

- **PostgreSQL** : <http://www.postgresql.org/> - dans le domaine public
- **MySQL** : <http://www.mysql.org/> - dans le domaine public
- **Oracle** : <http://www.oracle.com/> - de Oracle Corporation
- **IBM DB2** : <http://www-306.ibm.com/software/data/db2/>
- **Microsoft SQL** : <http://www.microsoft.com/sql/>
- **Sybase** : <http://www.sybase.com/linux>
- **Informix** : <http://www-306.ibm.com/software/data/informix/>

Modélisation d'un SI - notions élémentaires

2 Modélisation d'un SI - notions élémentaires

- Motivations
- Les différents outils d'analyse
- Le diagramme de classes UML

Modélisation d'un SI - notions élémentaires

2 Modélisation d'un SI - notions élémentaires

- **Motivations**
- Les différents outils d'analyse
- Le diagramme de classes UML

Motivations

Le but de cette section est de donner des outils élémentaires pour modéliser une base de données :

- pourquoi modélise-t-on ?
- les différents modèles pour les Systèmes d'Information (SI)
- les diagrammes de classes UML

Pourquoi modéliser ?

Définition

Un modèle est une représentation abstraite et simplifiée (c'est-à-dire qui exclut certains détails), d'une entité^a du monde réel en vue de le décrire, de l'expliquer ou de le prévoir.

a. phénomène, processus, système, etc.

Objectifs :

- réduire la complexité d'un phénomène en éliminant les détails non-significatifs
- permettre la compréhension et la prédiction du phénomène modélisé

Les limites du phénomène modélisé dépendent des objectifs du modèle.

Pourquoi modéliser ?

Exemples de modèles :

- **Modèle météorologique** : prévoir les conditions climatiques à partir d'observations satellites
- **Modèle économique** : simuler l'évolution de cours boursiers en fonction d'hypothèses macro-économiques (évolution du chômage, taux de croissance, ...)
- **Modèle démographique** : définir la composition d'un panel d'une population et son comportement,
- **Plans** : implantation du bâtiment, plans généraux du bâtiment et de sa structure, plans détaillés par pièce, plans des câblages électriques, plans d'écoulements des eaux, *etc.*

Pourquoi modéliser ?

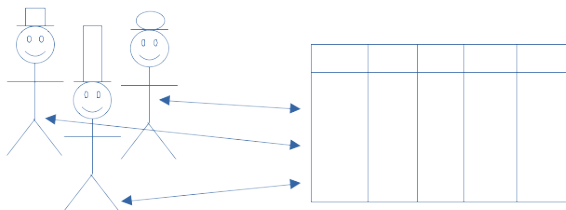
L'intérêt de modéliser un système avant sa réalisation :

- mieux **comprendre** son fonctionnement
- **maîtriser** sa complexité et d'assurer sa cohérence
- **communiquer** avec tous à l'aide un langage commun et précis
- bonus : réduction des coûts et des délais : certaines plateformes de modélisation savent maintenant exploiter les modèles pour faire de la génération de code.

Notre objectif dans ce module : modéliser des données (et leurs structures).

Pourquoi modéliser ?

Première approche d'une modélisation sur un exemple : gestion des étudiants en BUT INFO



- Identifier les informations intéressantes

Modélisation d'un SI - notions élémentaires

2 Modélisation d'un SI - notions élémentaires

- Motivations
- Les différents outils d'analyse
- Le diagramme de classes UML

Les différents outils d'analyse

On peut faire la distinction, d'une part, entre :

- composition : les méthodes ascendantes qui consistent à construire un logiciel par composition à partir de modules existants
- décomposition : les méthodes descendantes qui décomposent récursivement le système jusqu'à arriver à des modules programmables simplement

Les différents outils d'analyse

mais on peut aussi faire la distinction entre la stratégie

- fonctionnelle (dirigée par le traitement), également qualifiée de structurée
- orientée objet où un système est un ensemble d'objets interagissants

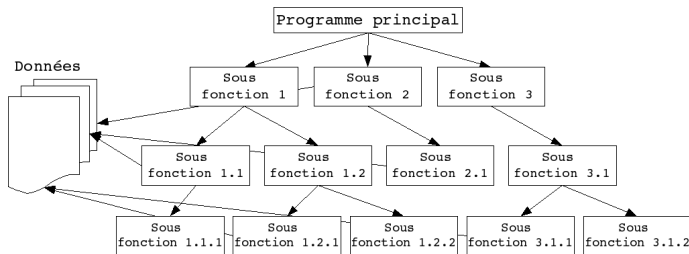


Figure – Représentation graphique d'une approche fonctionnelle [L. Audibert, UML2]

Les différents outils d'analyse

MERISE

MERISE (Méthode d'Étude et de Réalisation Informatique pour les Systèmes d'Entreprise) est certainement le langage de spécification le plus répandu dans la communauté de l'informatique des systèmes d'information.

- un de ses concepts clés : séparation des données (la statique) et des traitements (la dynamique)
- une modélisation des données en entités et en associations
- une démarche (méthode) d'analyse et de conception

Trois niveaux de représentation des données :

- Niveau conceptuel : modèle conceptuel des données (MCD)
- Niveau logique : modèle logique des données (MLD)
- Niveau physique : modèle physique des données (MPD)

Modélisation d'un SI - notions élémentaires

2 Modélisation d'un SI - notions élémentaires

- Motivations
- Les différents outils d'analyse
- Le diagramme de classes UML

L'approche orientée objet

L'approche considère le logiciel comme

- une collection d'objets dissociés,
- identifiés,
- et possédant des caractéristiques.

Une caractéristique est soit

- un attribut (une donnée caractérisant l'état de l'objet),
- une entité comportementale de l'objet (une méthode).

La fonctionnalité du logiciel émerge alors de l'interaction entre les différents objets qui le constituent.

L'approche orientée objet

A l'IUT de Lannion, UML est utilisé pour faire de la Conception Orientée Objet (COO) de logiciels.

Le travail conceptuel nécessaire est conséquent :

- définition des classes,
- de leurs relations,
- des attributs et méthodes,
- des interfaces, *etc.*

UML n'est pas une méthode, mais un langage graphique qui permet de représenter et de communiquer les divers aspects d'un système d'information

UML 2 dans sa généralité

UML 2.0 comporte ainsi treize types de diagrammes. Ils se répartissent en deux grands groupes :

Diagrammes structurels ou diagrammes statiques (UML Structure)

- diagramme de classes (Class diagram)
- diagramme d'objets (Object diagram)
- diagramme de composants (Component diagram)
- diagramme de déploiement (Deployment diagram)
- diagramme de paquetages (Package diagram)
- diagramme de structures composites (Composite structure diagram)

UML 2 dans sa généralité

UML 2.0 comporte ainsi treize types de diagrammes. Ils se répartissent en deux grands groupes :

Diagrammes comportementaux ou diagrammes dynamiques (UML Behavior)

- diagramme de cas d'utilisation (Use case diagram)
- diagramme d'activités (Activity diagram)
- diagramme d'états-transitions (State machine diagram)
- diagrammes d'interaction (Interaction diagram)
 - ▶ diagramme de séquence (Sequence diagram)
 - ▶ diagramme de communication (Communication diagram)
 - ▶ diagramme global d'interaction (Interaction overview diagram)
 - ▶ diagramme de temps (Timing diagram)

UML pour la BDD

Nous utiliserons uniquement le diagramme de classes pour modéliser les données.

L'objectif sera, à partir de l'énoncé des besoins, d'identifier

- les différentes collections d'objets intervenant dans le logiciel, que nous regrouperons en classes
- les relations entre ces différentes collections d'objets, que nous qualifierons en associations
- pour chaque classe ou association, ses caractéristiques : un attribut (une donnée) ou une méthode (fonction)³

3. Nous nous concentrerons sur les données pour le cours de BDD

Le diagramme de classes

Les classes

Définition (classe)

Une classe est un type de données abstrait qui précise des caractéristiques (attributs et méthodes) communes à toute une famille d'objets et qui permet de créer (instancier) des objets possédant ces caractéristiques.

Quelques exemples : Une instance est une concrétisation d'un concept abstrait.

- la Ferrari Enzo qui se trouve dans votre garage est une instance du concept abstrait Automobile
- l'amitié qui lie Jean et Marie est une instance du concept abstrait Amitié

Le diagramme de classes

Les classes

D'autres exemples : Une classe est un concept abstrait représentant des éléments variés comme :

- des éléments concrets (ex. : des avions),
- des éléments abstraits (ex. : des commandes de marchandises ou services),
- des composants d'une application (ex. : les boutons des boîtes de dialogue),
- des structures informatiques (ex. : des tables de hachage),
- des éléments comportementaux (ex. : des tâches), etc.

Le diagramme de classes

Caractéristiques d'une classe

Une classe définit un ensemble d'objets dotés de **caractéristiques communes**.

État d'un objet : ce sont les *propriétés (attributs et terminaisons d'association*⁴) qui décrivent l'état d'un objet,

Comportement d'un objet : ce sont les méthodes qui décrivent les éléments individuels d'un comportement. Ce sont des fonctions qui peuvent prendre des valeurs en entrée et modifier les attributs ou produire des résultats.

4. dont on reparlera un peu plus loin

Le diagramme de classes

Représentation graphique d'une classe

Une classe est représentée par un rectangle divisé en trois à cinq compartiments. On s'intéressera essentiellement aux 2 premiers ici.

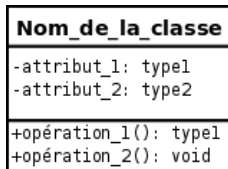


Figure – Représentation graphique d'une classe [L. Audibert, UML2]

- Le premier indique le nom de la classe,
- le deuxième ses attributs,
- et le troisième ses méthodes.

Les attributs dérivés (calculés à partir d'autres attributs et de formules de calcul) sont symbolisés par l'ajout d'un " / " devant leur nom

Le diagramme de classes

Exemples graphiques de classes

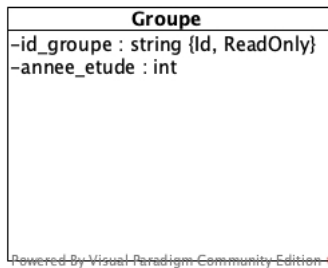
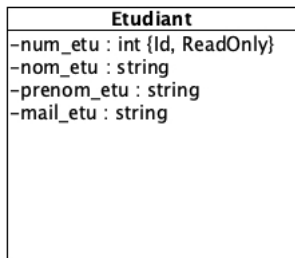


Figure – Exemple de deux classes

Le diagramme de classes

Relations entre classes

Définition

Une association est une relation entre deux classes (association binaire) ou plus (association n-aire), qui décrit les connexions entre leurs instances.

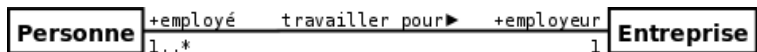


Figure – Exemple d'association binaire [L. Audibert, UML2]

Une association est caractérisée par un **nom**, une **multiplicité** (ou cardinalité) sur chaque terminaison et éventuellement **un rôle** sur les terminaisons.

Le diagramme de classes

Multiplicités d'une association

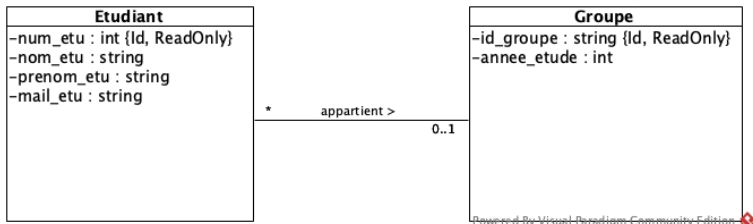
La multiplicité associée à une terminaison d'association déclare le nombre d'objets susceptibles d'occuper la position définie par la terminaison d'association.

Voici quelques exemples de multiplicité :

- exactement un : 1 ou 1..1 ;
- plusieurs : * ou 0..* ;
- au moins un : 1..* ;
- de un à six : 1..6.

Le diagramme de classes

Multiplicités d'une association - Exemples



- `*` (ou `0..*`) du côté de *Etudiant* signifie qu'un groupe peut compter 0 ou plusieurs étudiants en son sein,
- `0..1` du côté de *Groupe* signifie qu'un étudiant peut n'être inscrit dans aucun groupe, et appartenir à un groupe et un seul.

Le diagramme de classes

Multiplicités d'une association - Exemples

Etudiant

num_etu	nom_etu	...
E01	Martin	...
E02	Le Penven	...
E03	Guillot	...
E04	Fernandez	...

Groupe

id_groupe	annee_etude
F1	1
F2	1
A1	2
A2	2

Appartient

num_etu	id_groupe
E01	F1
E02	F1
E03	F2

Les valeurs présentées ici dans *Appartient* respectent bien les multiplicités.

Mais si on avait affecté un groupe à l'étudiant numéro *E04*, les multiplicités seraient toujours respectées.

Le diagramme de classes

Classe-association

Parfois, une association doit posséder des propriétés.

Ces propriétés n'appartiennent ni à une classe ni à l'autre, mais à l'association entre les deux seulement.

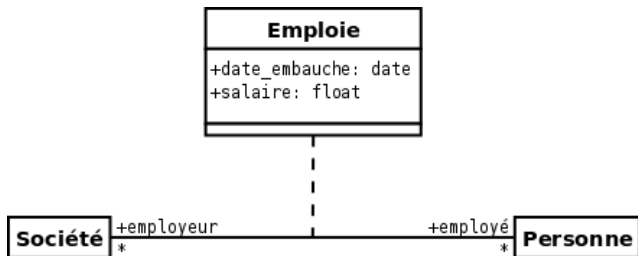


Figure – Exemple de classe-association [L. Audibert, UML2]

Ici, les deux propriétés *date_embauche* et *salaire* n'appartiennent ni à la société, qui peut employer plusieurs personnes, ni aux personnes, qui peuvent avoir plusieurs emplois.

Les bases du Modèle Relationnel (MR)

3 Les bases du Modèle Relationnel (MR)

- Éléments du modèle relationnel
- Schémas relationnels de base et Déclarations de schéma (LDD)

Introduction

Le but de cette section est de donner une vue concrète rapide sur :

- ce qu'est une base de données ;
- ce qu'est une base de données relationnelle, en particulier ;
- ce qu'est un *Système de Gestion de Bases de Données* (SGBD, DBMS en Anglais) ;
- ce que fait un SGBD ;
- comment un SGBD relationnel fait ce que fait un SGBD

Le Modèle Relationnel

Historique

Définition

Une base de données relationnelle est une base de données structurée suivant les principes de l'algèbre relationnelle.

- Père des BDD relationnelles : Edgar Frank Codd (1970)
- Utilisation de la théorie des ensembles et de la logique des prédicats du premier ordre
- Objectifs :
 - ▶ éviter la redondance des données
 - ▶ assurer l'intégrité des données
 - ▶ assurer l'indépendance de la structure de la base de données avec sa mise en oeuvre physique

Le Modèle Relationnel

Historique

- Article de Codd en 1970 :
 - ▶ stocker des données hétérogènes dans des tables
 - ▶ établir des relations entre elles
- L'idée est maintenant répandue, mais était considérée comme une curiosité intellectuelle à l'époque.
- Un premier prototype de Système de gestion de bases de données relationnelles (SGBDR) a été construit dans les laboratoires d'IBM
- Depuis les années 80, cette technologie a mûri et a été adoptée par l'industrie.
- En 1987, le langage SQL, qui étend l'algèbre relationnelle, a été standardisé.

C'est dans ce type de modèle que se situe ce cours de base de données.

Les bases du Modèle Relationnel (MR)

3 Les bases du Modèle Relationnel (MR)

- Éléments du modèle relationnel
- Schémas relationnels de base et Déclarations de schéma (LDD)

Éléments du modèle relationnel

De façon informelle, on peut définir le modèle relationnel de la manière suivante :

- il est composé de trois briques de base : type, attribut, relation ;
- les données sont organisées sous forme de "tables" à deux dimensions, encore appelées relations, dont les lignes sont appelées **n-uplets** (ou **tuples** en Anglais) ;
- les données sont manipulées par des opérateurs de l'algèbre relationnelle ;
- l'état cohérent de la base est défini par un ensemble de contraintes d'intégrité.

Éléments du modèle relationnel

Attribut

Définition

Un **attribut** est un identificateur (un nom) décrivant une information stockée dans une base.

Exemples : l'âge d'une personne, le nom d'une personne, le numéro de sécurité sociale.

Éléments du modèle relationnel

Type

Définition

*Le **type d'un attribut** est l'ensemble, fini ou infini, de ses valeurs possibles. Une sémantique (sens) est associé au type.*

Exemples : l'attribut *numéro de sécurité sociale* a pour type l'ensemble des combinaisons de quinze chiffres et *nom* a pour type l'ensemble des combinaisons de lettres (une combinaison comme cette dernière est généralement appelée chaîne de caractères ou, plus simplement, chaîne).

Éléments du modèle relationnel

Rappels mathématiques

Définition

Produit cartésien de deux ensembles E et F : l'ensemble noté $E \times F$ des couples (a, b) où a est un élément de E , et b un élément de F

Par exemple, si $Pays = \{France, Italie, Japon\}$ et $Devise = \{euro, yen\}$, alors

$$Pays \times Devise = \{(France, euro), (France, yen), (Italie, euro), (Italie, yen), (Japon, euro), (Japon, yen)\}$$

- $E \times F \neq F \times E$ (non-comutatif)
- $Card(E \times F) = Card(E) \cdot Card(F)$

Éléments du modèle relationnel

Relation

Définition

Relation : un sous-ensemble du produit cartésien d'une liste finie de n domaines. Elle est caractérisée par un nom.

Exemples :

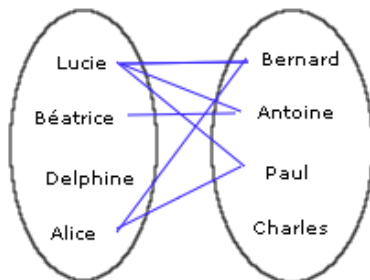
- Quels sont les sports pratiqués par les Français : établir une relation de l'ensemble des Français dans l'ensemble des sports possibles.
- $DeviseCourante = \{(france, euro), (Italie, euro), (Japon, yen)\}$

Une relation est représentée sous la forme d'un tableau à deux dimensions dans lequel les n attributs correspondent aux titres des n colonnes.

Éléments du modèle relationnel

Relation - Représentations graphiques

- représentation sagittale

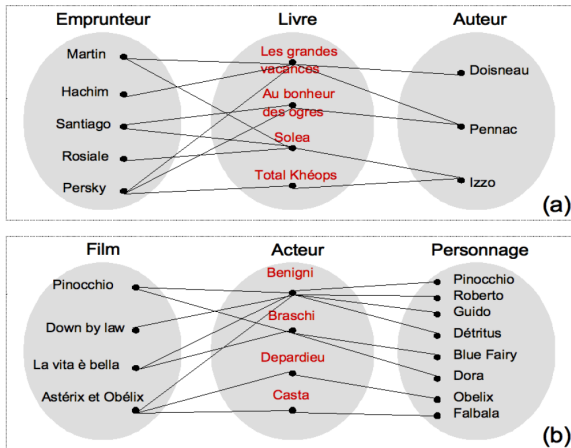


- représentation par matrice de coïncidence

Éléments du modèle relationnel

Relation - Représentations graphiques

Relations ternaires (tiré de [5])



Éléments du modèle relationnel

Schéma de relation

Définition

Un **schéma de relation** précise le nom de la relation ainsi que la liste des attributs avec leurs types.

Définition

Le **type d'une relation** précise la liste des attributs de la relation avec leurs types.

Même si on adoptera souvent la représentation tabulaire, table et relation ne sont pas synonymes.

Éléments du modèle relationnel

Degré, Occurrence, Cardinalité

Définition

Le **degré d'une relation** est le nombre de ses attributs.

Définition

Une **occurrence**, ou ***n*-uplet**, ou **tuple**, est un élément de l'ensemble figuré par une relation.

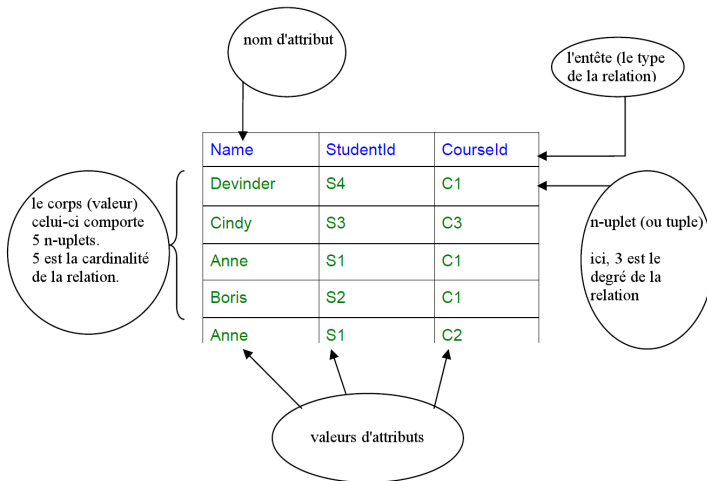
Autrement dit, une occurrence est une ligne dans la représentation tabulaire d'une relation.

Définition

La **cardinalité d'une relation** est le nombre de ses occurrences.

Éléments du modèle relationnel

Anatomie d'une relation



Éléments du modèle relationnel

table et relation

Correspondance des terminologies entre modèle relationnel et "représentation anatomique" (tabulaire) :

- relation : table
- n-uplet : ligne
- attribut : colonne

Nous utiliserons une terminologie 'table', mais il ne faut pas perdre de vue les différences entre relation et table (voir en fin de de chapitre).

Éléments du modèle relationnel

Gestion de l'intégrité des structures

- un ensemble de règles d'intégrité structurelle permet de faciliter la gestion de la cohérence des données.
- Ces règles sont énoncées par des assertions sur les relations et les attributs.
- Notion principale : **clef**.
- Mais aussi : contrôle des données manquantes et des valeurs d'attribut.
- De façon générale : extrêmement important de déclarer toutes les contraintes. Permet ensuite de se reposer sur le SGBDR pour la cohérence.

Éléments du modèle relationnel

Rappels mathématiques

Définition

Une application (ou fonction) f est la donnée de :

- *l'ensemble de départ E et l'ensemble d'arrivée F ;*
- *une relation associant à chaque élément x de l'ensemble de départ un et un seul élément de l'ensemble d'arrivée, que l'on appelle image de x par f et que l'on note $f(x)$.*

On dit alors que f est une application de E dans F (notée $f : E \rightarrow F$), ou encore une application à arguments dans E et valeurs dans F .

Éléments du modèle relationnel

Clé candidate

Définition

*Une **clé d'une relation** est un ensemble minimal des attributs de la relation dont les valeurs identifient à coup sûr une occurrence.*

La valeur d'une clé est donc distincte pour tous les n-uplets de la relation.

Il y a donc une fonction définie qui, pour une valeur de la clé, donne une et une seule valeur de chaque attribut de la relation.

Éléments du modèle relationnel

Clé candidate

Règle

Toute relation a au moins une clé et peut en avoir plusieurs.

- jamais deux n-uplets identiques au sein d'une relation
- pas forcément le même nombre d'attributs pour les différentes clés d'une relation
- une clé peut être formée d'un attribut arbitraire, utilisé à cette seule fin

Éléments du modèle relationnel

Propriétés de la relation

Une relation :

- a des attributs identifiables (de noms tous distincts)
- possède une clé au moins et n'a donc pas deux n-uplets identiques
- a des attributs tous valués (pas de NULL)
- a des attributs dont la valeur est connue (pas de UNKNOWN)

On constatera que certaines implantations du modèle relationnel ne respectent pas ces règles.

Éléments du modèle relationnel

Table $\neq$ Relation

Donnez trois raisons pour lesquelles cette table n'est pas une relation :

A	B	A
1	2	3
4		5
6	7	8
9	9	?
1	2	3

Éléments du modèle relationnel

Clé étrangère - Contrainte de référence

- identifier/référencer un n-uplet : utiliser sa valeur d'une de ses clés.

Exemple : ('Kevin', 'Spacey', 5), n-uplet de la relation Acteur(Nom, Prénom, numéro), est référencé par le numéro 5 :
Acteur[numéro=5] le désigne exactement.

- La valeur des attributs référentiels dans le n-uplet référent est la valeur de la clé dans le n-uplet référencé.

Éléments du modèle relationnel

Clé étrangère - Contrainte de référence

Exemple

- Pays(nom, devise, capitale, superficie)
- Devise(numéro, nbpieces, nbbillets, nom)
- Le Pays référence sa monnaie via la clé de la relation Devise : les valeurs de l'attribut devise de la relation Pays sont les valeurs de l'attribut numéro de la relation Devise.
- Ex. : Devise(4, 8, 8, 'euro'), Pays('France', 4, 'Paris', 544435)

Éléments du modèle relationnel

Clé étrangère - Contrainte de référence

Définition

Une **clé étrangère dans une relation** est formée d'un ou plusieurs attributs qui constituent une clé candidate dans une autre relation.

- Une contrainte référentielle exprime un lien obligatoire entre deux relations.
- Une relation possède une ou plusieurs clés, et peut spécifier plusieurs clefs étrangères également.

Éléments du modèle relationnel

Clé étrangère - Contrainte de référence

- Un attribut peut être à la fois une clé et étrangère :
Film(numFilm, titre, année) : numFilm est une clé
Acteur(numActeur, nom, prénom) : numActeur est clé candidate
Casting(numFilm, numActeur, personnage) :
 - ▶ (numFilm, numActeur) est clé
 - ▶ numFilm fait référence à la clé numFilm dans la relation Film
 - ▶ numActeur fait référence à la clé numActeur dans la relation Acteur
- Un acteur peut jouer dans plusieurs films et un film peut contenir plusieurs acteurs : lien n-n

Éléments du modèle relationnel

Clé étrangère - Contrainte de référence

Une relation peut s'auto-référencer via une clef étrangère.

Ex.: `post(id, user, date, answer_to, content)` dans un système de gestion de messages (forum de discussion).

Retrouver le fil de la discussion :

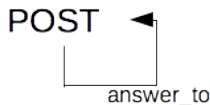
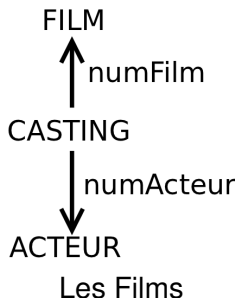
- (1, 'jean', '25/12/2015 00 :01', NULL, 'Joyeux Noël')
- (2, 'louis', '25/12/2015 00 :02', 1, 'Merci. A toi aussi !')
- (3, 'jean', '25/12/2015 00 :05', 2, 'Merci. La famille va bien ?')
- (4, 'jacques', '25/12/2015 00 :08', 1, 'Merci. Bonnes fêtes de fin d'année !')
- (5, 'bertrand', '25/12/2015 00 :09', 4, 'Merci. A toi aussi !')
- (6, 'jean', '25/12/2015 00 :10', 3, 'Au fait, tu as vu le message de Jacques ? Il est revenu de vacances.')

Éléments du modèle relationnel

Clé étrangère - Contrainte de référence

Graphe des contraintes d'intégrité référentielles :

- noeuds : les noms des relations
- arcs orientés : les clés référençantes



Messages dans un forum

Éléments du modèle relationnel

Contraintes référentielles et SGBDR

Importance pour le maintien de la cohérence lors des actions en écriture :

- Insertion d'un n-uplet dans une relation avec clé étrangère : le SGBD doit vérifier que la valeur donnée par ce n-uplet à la clé étrangère correspond bien à une valeur de clé primaire.
Même vérification en cas de modification d'une valeur de clé étrangère dans un n-uplet existant.
- Suppression d'un n-uplet référencé via une clé étrangère par un ensemble d'autres n-uplets : plusieurs options : supprimer "en cascade" tous les n-uplets référençant, ou donner la valeur NULL à l'attribut de clé étrangère ("set null") ou refuser la suppression ("restrict").
Par défaut, le SGBD ne fait rien ("no action").

Éléments du modèle relationnel

Schéma et base de données relationnels

Définition

Un **schéma relationnel** est constitué par l'ensemble des schémas de relations.

Définition

Une **base de données relationnelle** est constituée par l'ensemble des n -uplets des différentes relations du schéma relationnel. On parle aussi de la **valeur de la base**.

Définition

De même, on désignera par **valeur d'une relation**, l'ensemble des n -uplets de la relation.

Les bases du Modèle Relationnel (MR)

3 Les bases du Modèle Relationnel (MR)

- Éléments du modèle relationnel
- Schémas relationnels de base et Déclarations de schéma (LDD)

Schéma relationnel

Déclaration d'une relation

Pour déclarer une relation, il faut :

- la nommer (donner son identificateur)
- nommer l'ensemble de ses attributs
- donner la liste des clés
- donner la liste des clés étrangères

Dans toute la suite, nous donnerons les traductions des différentes commandes

- en langage Tutorial D (conception, en P2)
- en langage SQL pour PostgreSQL (implantation, en P3)

Schéma relationnel

Déclaration d'une relation

Nous nous appuierons sur un exemple traitant de l'inscription d'étudiants à des cours.

enrolment (StudentId, CoursId)

- la clé est {StudentId, CoursId}
- les clés étrangères sont {StudentId}, {CoursId}

Nous pourrions le représenter de la façon suivante :

ENROLMENT
StudentID CourseID
key : {StudentId, CourseId}

Schéma relationnel

Déclaration et destruction d'une relation - Tutorial D

Pour créer une relation (en théorie) :

```
VAR enrolment BASE RELATION
    {StudentID  CHARACTER,
     CourseId   CHARACTER}
    KEY {StudentID, CourseId};
CONSTRAINT enrolment_fk1
    FOREIGN KEY {StudentId}
        REFERENCES Student{StudentId};
CONSTRAINT enrolment_fk2
    FOREIGN KEY {CourseId}
        REFERENCES Course{CourseId} ;
```

Pour détruire une relation :

```
DROP VAR enrolment ;
```


Schéma relationnel

Déclaration et destruction d'une relation - Tutorial D

Pour créer une relation, une autre version des clés étrangères (implantation Rel) :

```
VAR enrolment BASE RELATION
  {StudentID  SID,
   CourseId   CID}
  KEY {StudentID, CourseId};
  CONSTRAINT enrolment_fk1
    enrolment{StudentId} <= Student{StudentId};
  CONSTRAINT enrolment_fk2
    enrolment{CourseId} <= Course{CourseId} ;
```

Littéralement : l'ensemble des identifiants d'étudiants présents dans la relation `enrolment` est inclus dans l'ensemble des identifiants présents dans la relation `Student`.

Schéma relationnel

Déclaration d'une relation - Complément sur les clés étrangères

Il peut arriver (fréquemment) que les clés référencées et les clés référençantes n'aient pas le même nom.

Tutorial D : pas de support direct - renommage

...

```
FOREIGN KEY {StudentId}
REFERENCES Student{SId}
      RENAME {SId AS StudentId}
```

...

Schéma relationnel

Types

Définition

Un type est un ensemble nommé de valeurs. Les types sont utilisés pour contraindre les valeurs permises dans un cadre d'utilisation précis.

On pourra contraindre :

- les valeurs allouées à une variable
- les valeurs que peut prendre un paramètre
- les valeurs qu'un opérateur peut donner en sortie quand il est invoqué
- les valeurs qui peuvent apparaître pour un attribut particulier d'une relation

Schéma relationnel

Types - Tutorial D

[Cardelli et Wegner, 1985] Un des buts majeurs des systèmes de types est d'éviter toutes les questions embarrassantes à propos des représentations, et d'interdire des situations dans lesquelles ces questions peuvent apparaître.

On distingue les types de base suivants dans Tutorial D :

- CHARACTER (ou CHAR) pour les chaînes de caractères
- INTEGER (ou INT) pour les entiers
- RATIONAL pour les nombres rationnels, décrivant des valeurs contenant une virgule et une partie décimale comme 3.25, 1.0, 0.0, -7.935
- TUPLE pour les n-uplets
- RELATION pour les relations

Schéma relationnel

Types - Tutorial D

On peut aussi définir ses propres types. Par exemple, un type pour l'identifiant d'étudiant :

```
TYPE SID POSSREP SID { C CHAR
                        CONSTRAINT LENGTH(C) <=5
                        AND
                        STARTS_WITH(C, 'S')
                        AND
                        IS_DIGITS(SUBSTRING(C,1))
                        }
```

C'est une chaîne de caractères, de longueur 5, qui commence par un 'S', et dont le reste des caractères sont des chiffres.

Schéma relationnel

Type d'une relation - Tutorial D

Les relations ayant la même entête peuvent être considérées comme étant du même type.

La définition d'une relation doit donc contenir la spécification de son entête. En reprenant notre exemple :

```
RELATION { StudentId CHARACTER,  
           CourseID CHARACTER }
```

mais aussi

```
RELATION { CourseID CHARACTER,  
           StudentId CHARACTER }
```

car l'entête est un *ensemble* d'attributs.

Cette dernière expression pourrait être le type déclaré de la relation `enrolment` définie plus tôt. `enrolment` est considérée alors comme la *variable* d'une certaine base de données.

Schéma relationnel

Expression littérales - Tutorial D

Une expression littérale est une expression dans laquelle n'apparaît aucune variable.

Pour définir une relation on pourrait procéder comme suit (incomplet) :

```
RELATION {  
    TUPLE { StudentID S1, CourseId C1 },  
    TUPLE { StudentID S1, CourseId C2 },  
    TUPLE { StudentID S2, CourseId C1 },  
    TUPLE { StudentID S3, CourseId C3 },  
    TUPLE { StudentID S4, CourseId C1 }  
}
```

Il n'est pas raisonnable de s'attendre à ce que les symboles comme S1 et C1 soient reconnus tels quels.

Il faut se conformer au type CHARACTER.

Schéma relationnel

Expression littérales - Tutorial D

L'expression littérale suivante est correcte

```
RELATION {  
    TUPLE {StudentID CHARACTER('S1'),  
           CourseId  CHARACTER('C1')} ,  
    TUPLE {StudentID CHARACTER('S1'),  
           CourseId  CHARACTER('C2')} ,  
    TUPLE {StudentID CHARACTER('S2'),  
           CourseId  CHARACTER('C1')}  
}
```

Remarquez l'absence d'ambigüité sur la correspondance de 'position' des attributs et des valeurs dans cette expression.

Schéma relationnel

Peuplement de la base - Tutorial D

Peuplement d'une relation : action de fournir des n-uplets pour la relation.

Cela peut se faire par affectation simple.

```
enrolment = RELATION {  
    TUPLE {StudentID CHARACTER('S1'),  
           CourseId  CHARACTER('C1')} ,  
    TUPLE {StudentID CHARACTER('S1'),  
           CourseId  CHARACTER('C2')} ,  
    TUPLE {StudentID CHARACTER('S2'),  
           CourseId  CHARACTER('C1')}  
}
```

Schéma relationnel

Peuplement de la base - Tutorial D

Ajout de n-uplets par **insertion** :

```
INSERT produit
RELATION {
    TUPLE {
        no_prod    CHAR('P8'),
        qte_stock  INT(150),
        categ      CHAR('s'),
        tva        INT(1)
    }
};
```

Opérateurs et Langage de Manipulation de Données (LMD)

4 Opérateurs et Langage de Manipulation de Données (LMD)

- Opérateurs : introduction
- Opérateurs ensemblistes et LMD
- Opérateurs relationnels simples et LMD
- LMD complexe

Opérateurs et Langage de Manipulation de Données (LMD)

4 Opérateurs et Langage de Manipulation de Données (LMD)

- Opérateurs : introduction
- Opérateurs ensemblistes et LMD
- Opérateurs relationnels simples et LMD
- LMD complexe

Opérateurs : pour quoi faire ?

- Jusqu'à maintenant, on a vu comment exprimer et déclarer des schémas relationnels.
- Ceci doit conduire à une base de données et à son exploitation.
- Il faut donc des outils pour :
 - ▶ Consulter des données dans la base
 - ▶ Insérer des valeurs dans la base (la *peupler*) : déjà vu
 - ▶ Supprimer ou modifier des valeurs dans la base

Opérateurs : une base formelle

- un support mathématique cohérent : l'algèbre relationnelle
- langage descriptif, plutôt qu'impératif
- opérations possibles sur les relations pour obtenir de nouvelles relations
- dans ce cours : Tutorial D (P2) et SQL (P3)

Opérateurs : les familles

On peut distinguer trois familles d'opérateurs relationnels :

- **Les opérateurs unaires (Restriction, Projection)**

les opérateurs les plus simples, ils permettent de produire une nouvelle relation à partir d'une autre relation

- **Les opérateurs binaires ensemblistes (Union, Intersection Différence)**

opérateurs permettant de produire une nouvelle relation à partir de deux relations de même degré et de même domaine

- **Les opérateurs binaires ou n-aires (Produit cartésien, Jointure, Division)**

permettant de produire une nouvelle relation à partir de deux ou plusieurs autres relations

Opérateurs : Exemple

CLIENT

no_cli	nom	prénom
C100	PRATT	Jean
C200	RIBEIRA	Pierre
C300	KLEIN	André

PROSPECT

nu_pro	nomp	prenomp
C100	PRATT	Jean
C897	LOISEL	Pierre
C567	COTHIAS	Arthur

LI_CO

no_cli	no_prod	qté
C100	P1	150
C100	P5	250
C200	P3	350
C100	P3	100

PRODUIT

no_prod	qte_stock	categ	tva
P1	1500	f	2
P3	130	f	1
P5	1000	s	1

Opérateurs et Langage de Manipulation de Données (LMD)

4 Opérateurs et Langage de Manipulation de Données (LMD)

- Opérateurs : introduction
- **Opérateurs ensemblistes et LMD**
- Opérateurs relationnels simples et LMD
- LMD complexe

Opérateurs ensemblistes

Pour intervenir dans un opérateur ensembliste, deux relations doivent être *union-compatibles*.

Définition

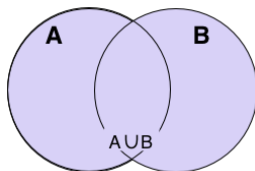
Deux relations sont union-compatibles s'il existe une correspondance un à un des types de leur attributs.

En réalité, ces deux relations doivent avoir le même type (même entête).

Opérateurs ensemblistes - Union

Tutorial D

```
s = r1 UNION r2;
```



- s , $r1$ et $r2$ sont de même type
- la valeur de s est l'ensemble des n -uplets, où chacun d'entre eux est soit un n -uplet de $r1$, soit un n -uplet de $r2$ (ou des deux).

Opérateurs ensemblistes - Union

Exemple

PRODUIT1

no_prod	qte_stock	categ	tva
P1	1500	f	2
P3	130	f	1

PRODUIT2

no_prod	qte_stock	categ	tva
P1	1500	f	2
P5	1000	s	1

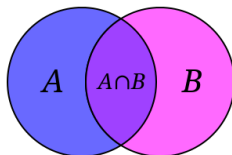
PRODUIT1 UNION PRODUIT2

no_prod	qte_stock	categ	tva
P1	1500	f	2
P3	130	f	1
P5	1000	s	1

Opérateurs ensemblistes - Intersection

Tutorial D

```
s = r1 INTERSECT r2;
```



- s , $r1$ et $r2$ sont de même type
- la valeur de s est l'ensemble des n -uplets, où chacun d'entre eux est un n -uplet de $r1$ et de $r2$.

Opérateurs ensemblistes - Intersection

Exemple

PRODUIT1

no_prod	qte_stock	categ	tva
P1	1500	f	2
P3	130	f	1

PRODUIT2

no_prod	qte_stock	categ	tva
P1	1500	f	2
P5	1000	s	1

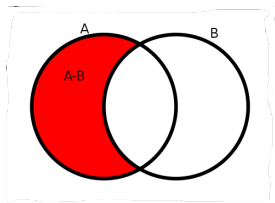
PRODUIT1 INTERSECT PRODUIT2

no_prod	qte_stock	categ	tva
P1	1500	f	2

Opérateurs ensemblistes - Différence

Tutorial D

```
s = r1 MINUS r2;
```



- s , $r1$ et $r2$ sont de même type
- la valeur de s est l'ensemble des n -uplets, où chacun d'entre eux est un n -uplet de $r1$ et n'est pas un n -uplet de $r2$.

Opérateurs ensemblistes - Différence

Exemple

PRODUIT1

no_prod	qte_stock	categ	tva
P1	1500	f	2
P3	130	f	1

PRODUIT2

no_prod	qte_stock	categ	tva
P1	1500	f	2
P5	1000	s	1

PRODUIT1 MINUS PRODUIT2

no_prod	qte_stock	categ	tva
P3	130	f	1

Opérateurs et Langage de Manipulation de Données (LMD)

4 Opérateurs et Langage de Manipulation de Données (LMD)

- Opérateurs : introduction
- Opérateurs ensemblistes et LMD
- **Opérateurs relationnels simples et LMD**
- LMD complexe

Tutorial D

$s = r \text{ WHERE } c;$

- s et r sont de même type
- c est une condition, expression possiblement vraie sur un ou plusieurs n-uplet de r
- la valeur de s est l'ensemble des n-uplets de r qui vérifient (chacun) la condition c .

Opérateurs relationnels - Restriction

Exemple :

```
s = prospect WHERE nu_pro = 'C897';
```

PROSPECT

nu_pro	nomp	prenomp
C100	PRATT	Jean
C897	LOISEL	Pierre
C567	COTHIAS	Arthur

prospect WHERE nu_pro = 'C897'

nu_pro	nomp	prenomp
C897	LOISEL	Pierre

Tutorial D

$s = r \{a_1, a_2, \dots, a_n\};$
 $= r \{ALL \text{ BUT } b_1, \dots, b_m\};$

- $\{a_1, a_2, \dots, a_n\}$ et $\{b_1, \dots, b_m\}$ sont des sous-ensembles disjoints de l'ensemble des attributs de r
- l'entête de s est le sous-ensemble de l'entête de r donné par $\{a_1, a_2, \dots, a_n\}$
- la valeur de s est l'ensemble des n -uplets de r dont on retire les attributs de $\{b_1, \dots, b_m\}$ (ou dont on n'affiche que les attributs de $\{a_1, a_2, \dots, a_n\}$).

Opérateurs relationnels - Projection

Exemple :

LI_CO

no_cli	no_prod	qté
C100	P1	150
C100	P5	250
C200	P3	350
C100	P3	100

LI_CO {no_prod}

no_prod
P1
P5
P3

Tutorial D

`s = r1 join r2 ;`

- $r1$ et $r2$ sont des relations dont les types ont une intersection nulle (pas d'attribut commun).
- le type de s est l'union des types de $r1$ et de $r2$.
- la valeur de s est l'ensemble des n -uplets de $r1$ combinés avec l'ensemble des n -uplets de $r1$.
- $Card(s) = Card(r1).Card(r2)$

Opérateurs relationnels - Produit cartésien

Exemple :

LI_CO

no_cli	no_prod	qté
C100	P1	150
C100	P5	250

PRODUIT

num_produit	qte_stock	categ	tva
P1	1500	f	2
P3	130	f	1
P5	1000	s	1

Opérateurs relationnels - Produit cartésien

Exemple : résultat

LI_CO JOIN PRODUIT

no_cli	no_prod	qté	num_produit	qte_stock	categ	tva
C100	P1	150	P1	1500	f	2
C100	P1	150	P3	130	f	1
C100	P1	150	P5	1000	s	1
C100	P5	250	P1	1500	f	2
C100	P5	250	P3	130	f	1
C100	P5	250	P5	1000	s	1

Tutorial D

`s = r1 join r2 ;`

- `r1` et `r2` sont des relations dont les types ont une intersection non-nulle (un attribut ou plusieurs en commun : même non, même type).
- le type de `s` est l'union des types de `r1` et de `r2`.
- la valeur de `s` est l'ensemble des n-uplets de `r1` combinés avec l'ensemble des n-uplets de `r2` qui ont une valeur qui correspond sur l'ensemble des attributs en commun.
- on appelle l'égalité des attributs en commun la condition de jointure

La jointure peut être vue comme un produit cartésien combiné à une restriction sur les attributs dans la clause de jointure.

Opérateurs relationnels - Jointure

Exemple :

LI_CO

no_cli	no_prod	qté
C100	P1	150
C100	P5	250

PRODUIT

no_prod	qte_stock	categ	tva
P1	1500	f	2
P3	130	f	1
P5	1000	s	1

Opérateurs relationnels - Jointure

Exemple : résultat

LI_CO JOIN PRODUIT

no_cli	no_prod	qté	no_prod	qte_stock	categ	tva
C100	P1	150	P1	1500	f	2
C100	P5	250	P5	1000	s	1

Opérateurs - Modification des données

Mise à jour

Mise à jour : recherche et remplacement de valeur

Tutorial D

```
UPDATE produit  
WHERE no_prod = CHAR ('P1') :  
{qte_stock := 1000};
```

Opérateurs - Modification des données

Suppression

Suppression de n-uplets

Tutorial D

```
DELETE produit  
WHERE no_prod = CHAR ('P1');
```

Opérateurs et Langage de Manipulation de Données (LMD)

4 Opérateurs et Langage de Manipulation de Données (LMD)

- Opérateurs : introduction
- Opérateurs ensemblistes et LMD
- Opérateurs relationnels simples et LMD
- LMD complexe

Compléments LMD : IN

Dans une condition (clause `WHERE`), l'opérateur `IN` permet de déterminer si la valeur à gauche de l'opérateur appartient à une des valeurs de la liste.

Exemples (Tutorial D) :

```
employees WHERE TUPLE{valstatut statut} IN  
RELATION{  
    TUPLE{valstatut DSTATUT('employe')},  
    TUPLE{valstatut DSTATUT('analyste')},  
    TUPLE{valstatut DSTATUT('vendeur')}}}
```

- `lvalstatut` est le nom de l'attribut en correspondance avec les 3 valeurs de la liste
- `statut` est une référence à la valeur de l'attribut testé dans `employees`

Compléments LMD - WITH

Une commande bien utile pour déclarer des variables relationnelles intermédiaires (et locales) :

```
WITH (pv := produit
      WHERE couleur = DCOULEUR('Vert'),
      ftpv := fourniture JOIN pv ) :
(ftpv WHERE qte >= 50){nofour} ;
```


Définition

Un **agrégat** (ou un groupe) est un ensemble de n -uplets qui peut être constitué d'une des façons suivantes :

- tous les n -uplets d'une relation ;
- certains n -uplets d'une relation après restriction ;
- les n -uplets d'une relation rassemblés sous un certain critère

et sur lequel on peut appliquer des fonctions ensemblistes.

La relation sur laquelle le ou les groupes est (sont) formé(s) peut provenir elle-même de plusieurs relations.

Agrégats

Exemples :

- tous les n-uplets de la relation `employees` ;
- les n-uplets de la relation `employees` qui satisfont la condition `sal>1500` ;
- Selon le critère `nodept`, 4 groupes de n-uplets peuvent être formés sur la relation `employees` :
 - ▶ un pour `nodept=22`,
 - ▶ un pour `nodept=56`,
 - ▶ un pour `nodept=35`,
 - ▶ et un pour `nodept=29`.

Fonctions d'agrégat

Plusieurs fonctions permettent de travailler sur les groupes.

En *Tutorial D*, on dispose de fonctions d'agrégat "hors requête" :

- **COUNT(*r*)** : compte le nombre de n-uplets de la relation ;
ex : COUNT(employees) ou
COUNT((employees WHERE nodept=22){noemp})
- **SUM(*r*, *x*)** : somme des valeurs de *x* dans la relation *r* ;
ex : SUM(employees WHERE noemp = DNOEMP('E1'), Sal)
- **MAX(*r*, *x*)** ou **MIN(*r*, *x*)** : maximum ou minimum des valeurs de *x* dans la relation *r* ;
ex : MAX(employees WHERE noemp = DNOEMP('E1'), Sal)
- **AVG(*r*, *x*)** = SUM(*r*, *x*) / COUNT(*r*) : moyenne des valeurs de *x* (numérique) dans *r* ;
ex : AVG(employees WHERE noemp = DNOEMP('E1'), Sal)

Il est également possible d'utiliser des fonctions d'agrégat dans les requêtes, auquel cas le nom de la relation n'est plus nécessaire (voir plus loin).

Fonctions d'agrégat

En *Tutorial D*, on aura besoin aussi de

Définition

```
EXTEND r: {x := f1, y := f2};
```

- où *f1* et *f2* sont des formules arbitraires, basées par exemple sur des fonctions d'agrégat
- permet d'étendre l'entête d'une relation avec ces formules.

Exemple :

```
EXTEND employes:  
    {revenu_mensuel := salaire + primes};
```

Agrégats - Exemple

EMPLOYEE

num_emp	nom_emp	salaire	nodept	statut
E1	Durand	2045.00	22	cadre
E2	Dupont	1561.00	22	ouvrier
E3	Bourhis	1601.00	22	ouvrier
E4	Le Guen	2789.00	29	cadre
E5	Le Bris	2233.00	29	cadre
E6	Le Bris	1345.00	35	ouvrier

On voudrait : Le salaire moyen par département

nodept	salaire_moyen
22	1735,66
29	2511,00
35	1345,00

Opérations sur des agrégats construits - Tutorial D

Définition

SUMMARIZE r BY $\{b_1, \dots, b_m\}$:
 $\{x := f_1, \dots, z := f_n\}$ où

- r *est une relation*
- $b_1, \dots, b_m$ *sont des noms d'attributs de r , sur lesquels on construit les regroupements (dans l'ordre de leur énumération)*
- $f_1, \dots, f_n$ *sont des formules arbitraires impliquant des fonctions d'agrégat*

Sur notre exemple, cela donnerait :

```
SUMMARIZE employees BY {nodept}:  
    {salaire_moyen := AVG(salaire)};
```

Opérations sur des agrégats construits - Tutorial D

Autre exemple : Calculer par département le nombre et le salaire moyen des employés qui sont ouvriers.

nodept	nombre_emp	salaire_moyen
22	2	1581.00
35	1	1345.00

Remarque : le département 29 n'apparaît pas car il n'y a pas d'ouvrier dans ce département.

Cela donnerait :

```
SUMMARIZE
  (employees WHERE statut = DSTATUT('ouvrier'))
BY {nodept}:
  {nombre_emp := COUNT(),
   salaire_moyen := AVG(salaire)};
```

Opérations sur des agrégats construits - Tutorial D

Autre exemple : Calculer par département le nombre et le salaire moyen des employés qui sont ouvriers. On voudrait que même les départements qui ne comptent pas d'ouvrier en leur sein apparaissent.

Il faut alors préciser que le regroupement doit se faire selon les valeurs de `nodept` qu'on trouve dans la relation `dept` et plus seulement dans `employees` pour lequel le département 29 n'apparaîtrait pas.

nodept	nombre_emp	salaire_moyen
22	2	1581.00
29	0	0.00
35	1	1345.00

Cela donnerait :

```
SUMMARIZE
```

```
(employees WHERE statut = DSTATUT('ouvrier'))
```

```
PER dept{nodept}:
```

```
{nombre_emp := COUNT(), sal_moy := AVG(salaire)};
```


Traitement de groupes faisant l'objet de conditions

- On peut spécifier une condition sur les groupes avant d'appliquer les fonctions de groupe.
- En Tutorial D, une restriction *a posteriori* sur un paramètre fonction du groupe suffit.
- La condition doit utiliser une fonction de groupe ; on peut y trouver aussi une requête (cf. la division dans la suite).

Traitement de groupes faisant l'objet de conditions - Tutorial D

Exemple : Trouver les départements (leurs numéros) où le nombre d'ouvriers est supérieur ou égal à 2 donnerait en Tutorial D

```
(  
  ( SUMMARIZE (employees  
               WHERE (statut=DSTATUT('ouvrier')))  
    BY {nodept}:  
       {nbemp := count() }  
  ) WHERE (nbemp >= 2)  
  ) {nodept}
```

Passage à PostgreSQL

5 Passage à PostgreSQL

- Environnement de travail
- LDD avec PostgreSQL
- LMD avec PostgreSQL

Passage à PostgreSQL

5 Passage à PostgreSQL

- Environnement de travail
- LDD avec PostgreSQL
- LMD avec PostgreSQL

Le SGBD : PostgreSQL

PostgreSQL est un système de gestion de base de données relationnelle et objet (SGBDRO).

C'est un outil libre disponible selon les termes d'une licence de type BSD.

Comme les projets libres Apache et Linux, PostgreSQL n'est pas contrôlé par une seule entreprise, mais est fondé sur une communauté mondiale de développeurs et d'entreprises.

C'est un système ouvert, extensible et puissant, et aussi bien adapté et intégré dans le monde du Web.

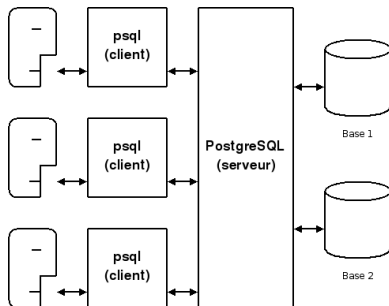
Le SGBD : PostgreSQL

- Organisation des données

- ▶ une (ou plusieurs) base de données par utilisateur
- ▶ autant de schémas que nécessaire dans chaque base

Un schéma est un *espace* dans lequel sont référencés des éléments (tables, vues, index, ...).

- schéma de fonctionnement client-serveur :



Connexion

Deux possibilités :

- client en ligne de commandes `psql`

```
psql [OPTIONS] ... [DBNAME [USERNAME]]
```

Options :

- ▶ `-d, --dbname=DBNAME`
- ▶ `-l, --list` : liste des bases
- ▶ `-U, --username=USERNAME`
- ▶ `-W, --password` : force la saisie de mot de passe
- ▶ `-H, --html` : format de sortie en HTML

Exemple :

```
psql -U delhay -h servbdd -W pg_delhay
```

- client avec emacs et son SQL mod

Utilisation

- **prompt** : `mabase==>`
- **saisie des requêtes** comme avec ORACLE
- `mabase==> \h` : obtenir de l'aide
- `mabase==> \q` : quitter
- **lire un fichier pour exécuter le SQL** :
 - ▶ une fois connecté :
`mabase==> \i monfichier.sql`
 - ▶ hors connexion, sous Unix :
`psql -U delhay -h servbdd -W -f monfichier.sql`
- **lire un fichier de données (CSV)**
`\COPY temps FROM '/home/users/temps.txt';`

Base, Schéma et tables

- Environnement de travail

```
DATABASE
  | - SCHEMA
    | - TABLE
```

Le schema est un moyen supplémentaire d'organiser ses tables dans quand on ne dispose que d'une seule base.

- En pratique

- ▶ `\c mabase` : pour changer de base (implicite à la connexion sur la base par défaut)
- ▶ `SELECT * FROM monschema.matable;`
- ▶ Possibilité de changer le chemin de recherche de schéma :
 - ★ `SHOW search_path ;`
 - ★ `SET search_path TO monschema ;`
 - ★ `SET SCHEMA 'monschema' ;`

Base, Schéma et tables

Quelques informations utiles :

- `SELECT version() ;`
- `SELECT current_database() ;`
- `SELECT current_user ;`
- `SELECT user ;`
- `\d` : liste des tables
- `\d <ma_table>` : caractéristiques de la table `ma_table`
- `\dn` : liste des schémas

Passage à PostgreSQL

5 Passage à PostgreSQL

- Environnement de travail
- LDD avec PostgreSQL
- LMD avec PostgreSQL

Rappel sur les éléments du modèle relationnel

table et relation

En SQL, des différences dues à l'implémentation :

- il est possible de définir des tables : pas de clé, plusieurs n-uplets identiques.
- si on définit une seule clé (candidate) pour une relation, celle-ci sera la clé primaire de la relation en SQL
- si on a plusieurs clés candidates dans une relation,
 - ▶ on désignera l'une d'entre-elles qui sera la clé primaire
 - ▶ la ou les autres se verront contraintes par une condition d'unicité
- il est possible d'avoir des attributs non-valués (NULL)
- il est possible d'avoir des attributs à valeur inconnue (UNKNOWN)

Toutefois, on essaiera de respecter au maximum le modèle relationnel.

Rappel sur les élément du modèle relationnel

Clé primaire

Définition

*La **clé primaire d'une relation** est une de ses clés (candidates).*

- On choisit une clef parmi les possibles pour identifier chaque n-uplet : c'est la clef primaire. Ce choix est guidé par la sémantique de la relation.
- Par convention, dans la littérature, on représente la clef primaire en la soulignant dans l'énoncé de la relation.

Schéma relationnel

Déclaration d'une relation

Nous nous appuierons à nouveau sur un exemple traitant de l'inscription d'étudiants à des cours.

`enrolment (StudentId, Name, CoursId)`

- **la clé candidate est** `{ (StudentId, CourseId) }`
- **les clés étrangères sont** `{ (StudentId), (CourseId) }`

Schéma relationnel

Déclaration et destruction d'une relation - SQL

Création :

```
CREATE TABLE enrolment (  
    StudentID    NUMERIC(3),  
    Name         VARCHAR(30),  
    CourseId     NUMERIC(3),  
    CONSTRAINT enrolment_pk  
        PRIMARY KEY (StudentID, CourseId),  
    CONSTRAINT enrolment_fk1 FOREIGN KEY (StudentId)  
        REFERENCES Student(StudentId),  
    CONSTRAINT enrolment_fk2 FOREIGN KEY (CourseId)  
        REFERENCES Course(CourseId)  
);
```

Suppression :

```
DROP TABLE enrolment ;
```

Schéma relationnel

Déclaration d'une relation - Complément sur les clés étrangères

Il peut arriver (fréquemment) que les clés référencées et les clés référençantes n'aient pas le même nom.

SQL : support direct des clés de noms différents

```
...  
    CONSTRAINT enrolment_fk1 FOREIGN KEY (StudentId)  
        REFERENCES Student (SId),  
...
```


Schéma relationnel

Types de base - SQL

En SQL (sous PostgreSQL), on distingue les types de base suivants :

- `CHAR [ (n) ]` : chaîne de caractères de longueur fixe, complété par des espaces, associé à `CHARACTER`
- `VARCHAR [ (n) ]` : chaîne de caractères de longueur variable avec limite, associé à `CHARACTER VARYING`
- `NUMERIC [ (p, s) ]`, associé à `DECIMAL [ (p, s) ]` : Nombre exact dont la précision peut être précisée
 - ▶ `p` (precision) : le nombre total de digits
 - ▶ `s` (scale) : le nombre de digits à droite du point décimal
- divers types numériques spécifiés : `DOUBLE PRECISION`, `FLOAT8`, `REAL`, `INTEGER`, `INT`, `SMALLINT`, `BIGINT`
- types dérivés : `%TYPE`, `%ROWTYPE`

Schéma relationnel de base

Modification de schéma - SQL

Commande ALTER TABLE : Ajout d'attributs

```
CREATE TABLE client(  
    no_client    VARCHAR(5),  
    nom          VARCHAR(32),  
    prenom       VARCHAR(32),  
    adresse      VARCHAR(150)  
);  
ALTER TABLE client ADD magasin_ref CHAR(3);
```

Commande ALTER TABLE : Suppression d'attributs

```
ALTER TABLE client DROP magasin_ref;
```

Schéma relationnel de base

Modification de schéma - SQL

Commande **ALTER TABLE** : Ajout de contraintes

```
ALTER TABLE client
    ADD CONSTRAINT pk_client PRIMARY KEY (no_client);
ALTER TABLE client
    ADD CONSTRAINT client_fk1
        FOREIGN KEY (magasin_ref)
        REFERENCES MAGASIN(num_magasin);
ALTER TABLE client
    ADD CONSTRAINT client_uk
    UNIQUE (nom, prenom);
```

Commande **ALTER TABLE** : Suppression de contraintes

```
ALTER TABLE client DROP CONSTRAINT fk1_client;
```

Passage à PostgreSQL

5 Passage à PostgreSQL

- Environnement de travail
- LDD avec PostgreSQL
- LMD avec PostgreSQL

Opérateurs : Exemple

CLIENT

no_cli	nom	prénom
C100	PRATT	Jean
C200	RIBEIRA	Pierre
C300	KLEIN	André

PROSPECT

nu_pro	nomp	prenomp
C100	PRATT	Jean
C897	LOISEL	Pierre
C567	COTHIAS	Arthur

LI_CO

no_cli	no_prod	qté
C100	P1	150
C100	P5	250
C200	P3	350
C100	P3	100

PRODUIT

no_prod	qte_stock	categ	tva
P1	1500	f	2
P3	130	f	1
P5	1000	s	1

Contenu d'une table

Pour obtenir le contenu d'une table, là où en Tutorial D il suffit d'appeler le nom d'une relation, en SQL on utilise `SELECT` qui est la commande de base d'interrogation :

```
SELECT *  
FROM <matable> ;
```

où `<matable>` est le nom de la table à interroger

Le `*` après le `SELECT` signifie qu'on désire 'tout' voir de la table (tous ses attributs).

Nous verrons que la commande `SELECT` est composée de différentes *clauses* : `WHERE`, `FROM`, `GROUP BY`, ...

Opérateurs ensemblistes

Pour intervenir dans un opérateur ensembliste, deux relations doivent être *union-compatibles*.

Définition

Deux relations sont union-compatibles s'il existe une correspondance un à un des types de leur attributs.

SQL

```
SELECT * FROM r1  
UNION  
SELECT * FROM r2;
```

- $r1$ et $r2$ sont des tables union-compatibles
- la valeur du résultat est l'ensemble des n-uplets, où chacun d'entre eux est soit un n-uplet de $r1$, soit un n-uplet de $r2$ (ou des deux).

Opérateurs ensemblistes - Union

Exemple

SELECT * FROM PRODUIT1

no_prod	qte_stock	categ	tva
P1	1500	f	2
P3	130	f	1

SELECT * FROM PRODUIT2

no_prod	qte_stock	categ	tva
P1	1500	f	2
P5	1000	s	1

SELECT * FROM PRODUIT1 UNION SELECT * FROM PRODUIT2

no_prod	qte_stock	categ	tva
P1	1500	f	2
P3	130	f	1
P5	1000	s	1

SQL

```
SELECT * FROM r1  
INTERSECT  
SELECT * FROM r2;
```

- $r1$ et $r2$ sont des tables union-compatibles
- la valeur du résultat est l'ensemble des n -uplets, où chacun d'entre eux est un n -uplet à la fois de $r1$ et de $r2$.

Opérateurs ensemblistes - Intersection

Exemple

SELECT * FROM PRODUIT1

no_prod	qte_stock	categ	tva
P1	1500	f	2
P3	130	f	1

SELECT * FROM PRODUIT2

no_prod	qte_stock	categ	tva
P1	1500	f	2
P5	1000	s	1

SELECT * FROM PRODUIT1

INTERSECT SELECT * FROM PRODUIT2

no_prod	qte_stock	categ	tva
P1	1500	f	2

SQL

```
SELECT * FROM r1  
EXCEPT  
SELECT * FROM r2;
```

- $r1$ et $r2$ sont des tables union-compatibles
- la valeur du résultat est l'ensemble des n -uplets, où chacun d'entre eux est un n -uplet de $r1$ et n'est pas un n -uplet de $r2$.

Opérateurs ensemblistes - Différence

Exemple

SELECT * FROM PRODUIT1

no_prod	qte_stock	categ	tva
P1	1500	f	2
P3	130	f	1

SELECT * FROM PRODUIT2

no_prod	qte_stock	categ	tva
P1	1500	f	2
P5	1000	s	1

SELECT * FROM PRODUIT1
EXCEPT SELECT * FROM PRODUIT2

no_prod	qte_stock	categ	tva
P3	130	f	1

SQL

```
SELECT *  
FROM  $r$   
WHERE  $c$ ;
```

- c est une condition, expression possiblement vraie sur un ou plusieurs n-uplets de r
- la valeur du résultat est l'ensemble des n-uplets de r qui vérifient (chacun) la condition c .

Cette opération est aussi parfois appelée *sélection*. Nous éviterons ce nom qui peut prêter à confusion avec le SELECT de SQL.

Opérateurs relationnels - Restriction

Exemple :

```
SELECT * FROM propect WHERE nu_pro = 'C897';
```

PROSPECT

nu_pro	nomp	prenomp
C100	PRATT	Jean
C897	LOISEL	Pierre
C567	COTHIAS	Arthur

```
SELECT * FROM propect WHERE nu_pro = 'C897';
```

nu_pro	nomp	prenomp
C897	LOISEL	Pierre

Opérateurs relationnels - Projection

SQL

```
SELECT a1, a2, ..., an  
FROM r ;
```

- $\{a1, a2, \dots, an\}$ est un sous-ensemble de l'ensemble des attributs de r
- la valeur du résultat est l'ensemble des n -uplets de r dont on n'affiche que les attributs de $\{a1, a2, \dots, an\}$.

Opérateurs relationnels - Projection

Exemple :

```
SELECT * FROM li_co ;
```

no_cli	no_prod	qté
C100	P1	150
C100	P5	250
C200	P3	350
C100	P3	100

```
SELECT no_prod FROM li_co ;
```

no_prod
P1
P5
P3
P3

Remarque : Apparition de doublons dans les résultats.

Opérateurs relationnels - Projection

ALL et DISTINCT

Résultat d'un opérateur algébrique

- un ensemble de vecteurs de dimension n (si c'est un n -uplet)
- ne comporte donc pas de doublons

Au contraire, le résultat d'un `SELECT` sélection peut comporter des doublons :

- Pour obtenir tous les n -uplets : `SELECT [ALL] attr1, attr2, FROM ...` (par défaut)
- Pour supprimer les doublons : `SELECT DISTINCT attr1, attr2, FROM ...` (voir l'exemple de la diapo précédente)

SQL

```
SELECT *  
FROM r1 CROSS JOIN r2 ;
```

- l'ensemble des attributs du résultat est l'union des ensembles des attributs de $r1$ et de $r2$.
- la valeur du résultat est l'ensemble des n-uplets de $r1$ combinés avec l'ensemble des n-uplets de $r2$.
- $Card(s) = Card(r1).Card(r2)$

Opérateurs relationnels - Produit cartésien

Exemple :

LI_CO

no_cli	no_prod	qté
C100	P1	150
C100	P5	250

PRODUIT

num_produit	qte_stock	categ	tva
P1	1500	f	2
P3	130	f	1
P5	1000	s	1

Opérateurs relationnels - Produit cartésien

Exemple : résultat

```
SELECT * FROM LI_CO CROSS JOIN PRODUIT;
```

no_cli	no_prod	qté	num_produit	qte_stock	categ	tva
C100	P1	150	P1	1500	f	2
C100	P1	150	P3	130	f	1
C100	P1	150	P5	1000	s	1
C100	P5	250	P1	1500	f	2
C100	P5	250	P3	130	f	1
C100	P5	250	P5	1000	s	1

Opérateurs relationnels - Jointure naturelle

C'est l'équivalent de la jointure de Tutorial D `JOIN`, où `r1` et `r2` sont des relations dont les types ont une intersection non-nulle (un attribut ou plusieurs en commun : même nom, même type).

Elle s'écrit en SQL :

```
SELECT *  
FROM r1 NATURAL JOIN r2;
```

Opérateurs relationnels - Equi-jointure interne

C'est le cas où on veut faire une jointure, mais qu'il n'y a pas forcément d'attribut commun. **SQL**

```
SELECT *  
FROM r1 INNER JOIN r2  
ON r1.att1 = r2.att2 ;
```

- la valeur du résultat est l'ensemble des n-uplets de $r1$ combinés avec l'ensemble des n-uplets de $r2$ qui ont une valeur) qui correspond (qui est égale) sur l'ensemble des attributs dans la condition de jointure exprimée dans la clause **ON**.
- on appelle l'égalité des attributs en commun la condition de jointure

L'équi-jointure interne peut être vue comme un produit cartésien combiné à une restriction d'égalité sur les attributs dans la clause de jointure.

Opérateurs relationnels - Equi-jointure interne

Exemple :

LI_CO

no_cli	no_prod	qté
C100	P1	150
C100	P5	250

PRODUIT

no_prod	qte_stock	categ	tva
P1	1500	f	2
P3	130	f	1
P5	1000	s	1

Opérateurs relationnels - Equi-jointure interne

Exemple : résultat

```
SELECT *  
FROM li_co INNER JOIN produit  
ON li_co.no_prod = produit.no_prod;
```

LI_CO JOIN PRODUIT

LI_CO			PRODUIT			
no_cli	no_prod	qté	no_prod	qte_stock	categ	tva
C100	P1	150	P1	1500	f	2
C100	P5	250	P5	1000	s	1

Remarque : Quand il y a ambiguïté sur la provenance de l'attribut, on préfixe le nom des attributs par le nom des tables.

Opérateurs - Modification des données

Mise à jour

Mise à jour : recherche et remplacement de valeur **SQL**

```
UPDATE produit  
SET qte_stock = 1000  
WHERE no_prod = 'P1';
```

```
UPDATE produit  
SET qte_stock = 1000;
```

Opérateurs - Modification des données

Suppression

Suppression de n-uplets

SQL

```
DELETE [FROM] produit  
WHERE no_prod = 'P1';
```

```
DELETE [FROM] produit;
```

Opérateurs - Modification des données

Insertion

Ajout de n-uplets

- **SQL** (Attention à l'ordre des valeurs : celui de la définition de la table)

```
INSERT INTO produit  
VALUES ('P8', 150, 's', 1);
```

- et d'autres méthodes ...

Compléments LMD SQL - IN

Dans une condition (clause `WHERE`), l'opérateur `IN` permet de déterminer si la valeur à gauche de l'opérateur appartient à une des valeurs de la liste.

Exemples (SQL) :

- `statut IN ('employe', 'analyste', 'vendeur')`
- `statut NOT IN ('cadre', 'PDG')`
- `(x, y, z) IN (SELECT a, b, c FROM t)`

Dans le dernier exemple, la liste est potentiellement dynamique (dépendante de la requête principale).

Compléments LMD SQL - WITH

Une commande bien utile pour déclarer des variables relationnelles intermédiaires (et locales) :

```
WITH ventes_regionales AS (  
    SELECT region, SUM(montant) AS ventes_totales  
    FROM commandes  
    GROUP BY region),  
meilleures_regions AS (  
    SELECT region  
    FROM ventes_regionales  
    WHERE ventes_totales >  
        (SELECT SUM(ventes_totales)/10  
         FROM ventes_regionales) )  
SELECT region, produit,  
    SUM(quantite) AS unites_produit,  
    SUM(montant) AS ventes_produit  
FROM commandes  
WHERE region IN (SELECT region  
                 FROM meilleures_regions)  
GROUP BY region, produit;
```

Définition

Un agrégat (ou un groupe) est un ensemble de n -uplets qui peut être constitué d'une des façons suivantes :

- *tous les n -uplets d'une relation ;*
- *certaines n -uplets d'une relation après restriction ;*
- *les n -uplets d'une relation rassemblés sous un certain critère et sur lequel on peut appliquer des fonctions ensemblistes.*

La relation sur laquelle le ou les groupes est (sont) formé(s) peut provenir elle-même de plusieurs relations.

Agrégats

Exemples :

- tous les n-uplets de la relation `employees` ;
- les n-uplets de la relation `employees` qui satisfont la condition `sal>1500` ;
- Selon le critère `nodept`, 4 groupes de n-uplets peuvent être formés sur la relation `employees` :
 - ▶ un pour `nodept=22`,
 - ▶ un pour `nodept=56`,
 - ▶ un pour `nodept=35`,
 - ▶ et un pour `nodept=29`.

Fonctions d'agrégat

En SQL on dispose des fonctions suivantes :

- **COUNT (*)** : compte le nombre de n-uplets de la table ou du groupe ;
ex :
`SELECT COUNT(noemp) FROM employes WHERE nodept=22;`
- **COUNT ([DISTINCT|ALL] expr)** : compte le nombre de valeur de expr non NULL ;
- **SUM ([DISTINCT|ALL] expr)** : somme des valeurs de expr ;
ex : `SELECT SUM(Sal) FROM WHERE noemp = E1;`
- **AVG ([DISTINCT|ALL] colonne)** : moyenne des valeurs de la colonne (numérique) ;
- **MAX ([DISTINCT|ALL] expr)** : maximum des valeurs de expr ;
- **MIN ([DISTINCT|ALL] expr)** : minimum des valeurs de expr ;

Toutes sauf COUNT (*) ignorent les valeurs NULL.

Règle

Dans la liste de projection, on ne doit pas trouver de mélange nom de colonne/fonction de groupe

Exemple :

```
SELECT nom, AVG(salaire) AS salaire_moyen  
FROM employes;
```

n'a aucun sens (en plus d'être fausse syntaxiquement).

Agrégats - Exemple

EMPLOYEE

num_emp	nom_emp	salaire	nodept	statut
E1	Durand	2045.00	22	cadre
E2	Dupont	1561.00	22	ouvrier
E3	Bourhis	1601.00	22	ouvrier
E4	Le Guen	2789.00	29	cadre
E5	Le Bris	2233.00	29	cadre
E6	Le Bris	1345.00	35	ouvrier

On voudrait : Le salaire moyen par département

nodept	salaire_moyen
22	1735,66
29	2511,00
35	1345,00

Opérations sur des agrégats construits - SQL

- Les agrégats peuvent être construits à l'aide de la clause `GROUP ... BY <clause de groupage>`.
- La clause de groupement est une liste de noms d'attributs.

La commande prend la forme suivante :

```
SELECT ... fonction(s) d'agrégat  
FROM ...  
[WHERE ... ]  
GROUP BY liste d'attributs ;
```

Opérations sur des agrégats construits - SQL

Le traitement appliqué est le suivant :

- la clause `WHERE` est d'abord appliquée et produit une table
- puis les groupes sont formés selon les valeurs de la liste d'attributs
- enfin sur chaque groupe sont appliquées les fonctions de groupe.

Règle

*La liste de projection peut contenir, outre les fonctions de groupe, les attributs figurant dans la clause `GROUP ... BY` et **UNIQUEMENT** ceux-ci.*

Opérations sur des agrégats construits - SQL

Exemples :

- Requête : Calcul du salaire moyen par département :

```
SELECT nodept, AVG(sal) SALAIRE_MOYEN  
FROM employes  
GROUP BY nodept ;
```

- Calculer par département le nombre et le salaire moyen des employés qui sont ouvriers :

```
SELECT nodept, COUNT(*), AVG(sal)  
FROM employes  
WHERE job = 'ouvrier'  
GROUP BY nodept ;
```

Opérations sur des agrégats construits - SQL

Exemples :

- La requête suivante est fausse (mélange dans la liste de projection).

```
SELECT nodept, nom, MIN(sal)
FROM employes
WHERE nom LIKE 'A%'
GROUP BY nodept ;
```

Traitement de groupes faisant l'objet de conditions

- On peut spécifier une condition sur les groupes avant d'appliquer les fonctions de groupe.
- En Tutorial D, une restriction *a posteriori* sur un paramètre fonction du groupe suffit.
- La condition doit utiliser une fonction de groupe ; on peut y trouver aussi une requête (cf. la division).
- En SQL, ceci se fait en utilisant dans le `GROUP BY` la clause `HAVING <condition>`.

Traitement de groupes faisant l'objet de conditions - SQL

Exemple : Trouver les départements où le nombre d'ouvriers est supérieur ou égal à 2

```
SELECT nodept
FROM employes
WHERE statut = 'ouvrier'
GROUP BY nodept
HAVING COUNT(*) >= 2 ;
```

- La table `employes` subit d'abord la sélection ;
- les n-uplets restant sont groupés selon la valeur de l'attribut `nodept` ;
- chaque groupe est retenu en sortie si son nombre de n-uplets est supérieur ou égal à 2.

Ordonnancement - SQL

Il est possible en SQL de préciser l'ordre de sortie des n-uplets à l'aide de la clause :

```
ORDER BY <nom_attribut> [ASC | DESC] [, ...  
      <nom_attribut> [ASC | DESC] ] *
```

- ASC : ordre ascendant, DESC ordre descendant ;
- les attributs doivent être dans la liste de projection ;
- les valeurs NULL apparaissent en tête ;
- `nom_attribut` peut être une expression.

Exemple :

```
SELECT nom, sal + comm  
FROM employes  
ORDER BY nom ;
```

Contrainte : plage de valeurs

Dans chaque cours, les points sont toujours compris entre 0 et 20.

```
CREATE TABLE resultats (  
    num_eleve    NUMBER(3),  
    num_cours    NUMBER(2),  
    points       INTEGER  
                CONSTRAINT plage_des_points  
                CHECK (points BETWEEN 0 AND 20)  
);
```

Contrainte : liste de valeurs

Les valeurs admises pour le `sexe` sont 'M', 'F', 'm', 'f' ou NULL.

```
CREATE TABLE personnel (  
    num      NUMBER(3),  
    nom      CHAR(20),  
    prenom   CHAR(20),  
    sexe     CHAR(1)  
        CONSTRAINT valeurs_admises_pour_sexe  
        CHECK (sexe IN ('M', 'F', 'm', 'f')  
              OR sexe IS NULL)  
);
```

La contrainte sur attribut (et non sur table) est admise quand la contrainte ne porte effectivement que sur l'attribut.

Bibliographie I



Pierre DELMAL.

SQL2-SQL3, Applications à ORACLE, 3ème Edition.

De Boeck Université, 2001, ISBN 2-8041-3561-6.



Jacques SIROUX.

Cours de Bases de Données - DUT Informatique



Laurent AUDIBERT.

Base de Données et Langage SQL.

[http ://laurent-audibert.developpez.com/Cours-BD](http://laurent-audibert.developpez.com/Cours-BD)



Laurent AUDIBERT.

UML2, de l'apprentissage à la pratique.

[https ://laurent-audibert.developpez.com/Cours-UML](https://laurent-audibert.developpez.com/Cours-UML)



Remi EYRAUD.

Base de Données.

[http ://www.lif.univ-mrs.fr/reyraud/BD](http://www.lif.univ-mrs.fr/reyraud/BD)

Enjeux

Illustration sur un cas pratique

Question pratique :

Pourquoi dois-je m'embêter avec toute cette théorie et ces connaissances à assimiler alors que je sais très bien manipuler un fichier, y stocker des informations et les y retrouver avec mon langage de programmation favoris ?

Autres question pratique :

Pourquoi dois-je m'embêter avec toute cette théorie et ces connaissances à assimiler alors que je sais très bien manipuler un tableur, y organiser des informations à stocker et les y retrouver ?

Enjeux

Illustration sur un cas pratique

A LIRE PAR VOUS MEME

Application de gestion d'une bibliothèque (tiré de [?])

- les livres de la bibliothèque :

- ▶ numéro de livre,
- ▶ un titre,
- ▶ un ou plusieurs auteurs
- ▶ un éditeur

- les emprunteurs :

- ▶ nom, prénom
- ▶ numero de téléphone
- ▶ adresse
- ▶ date de l'emprunt
- ▶ date de retour

Toutes les informations doivent être conservées pour garder un historique des emprunts.

Enjeux

Illustration sur un cas pratique

Approche naïve : utilisation d'un fichier à accès séquentiel (*cf.* cours d'algorithmique et programmation)

- un fichier comportant à l'origine une ligne par livre
- sur chaque enregistrement, on trouve titre, auteur, éditeur, numéro du livre
- à l'emprunt, on complète : nom, prénom, téléphone, adresse et date-emprunt
- au retour, on ajoute date-retour
- lors d'un nouvel emprunt, nouvelle ligne en dupliquant les données du livre et en ajoutant les données de la personne qui emprunte

Enjeux

Illustration sur un cas pratique

On peut voir ce fichier texte comme un tableau dont l'entête des colonnes seraient les suivantes :

titre	auteur	éditeur	n°livre	nom	prénom	téléphone	adresse	date-emprunt	date-retour
-------	--------	---------	---------	-----	--------	-----------	---------	--------------	-------------

L'application fonctionne maintenant depuis 10 ans.

Le nombre de personnes inscrites à la bibliothèque est relativement constant (bien que l'on constate un roulement) et de 5000 personnes en moyenne par an.

Un abonné emprunte en moyenne 5 livres par mois.

Enjeux

Illustration sur un cas pratique

Questions (1) :

- Quel est, approximativement, le nombre de lignes du fichier des données ?
- Quelle est la taille approximative du fichier sachant que chaque caractère occupe 1 octet et qu'une ligne contient, en moyenne, 150 caractères ?

Questions (2) :

- Lorsqu'un abonné emprunte un livre, le bibliothécaire saisit simplement le numéro du livre et le nom et le prénom de l'abonné. L'application se charge alors de parcourir le fichier pour rechercher les informations manquantes concernant le livre et l'abonné afin d'écrire, à la fin du fichier, la nouvelle ligne concernant l'emprunt. Dans le pire des cas, l'application doit parcourir tout le fichier. Supposons qu'un accès au fichier coûte 10ms, qu'une lecture de ligne coûte 6ms et qu'une recherche sur la ligne pour trouver le numéro du livre ou le nom et le prénom de l'abonné coûte 1ms. Quel est, dans le pire des cas, le temps mis par l'application pour compléter les informations saisies par le bibliothécaire ?

Questions (3) :

- Supposons qu'une personne est abonnée depuis l'origine de l'application. Elle prévient le bibliothécaire que son prénom est mal orthographié. Combien de lignes, approximativement, doivent être modifiées pour corriger cette erreur dans tout le fichier de données ?
- Lors de cette correction d'un prénom mal orthographié, comment distinguer les homonymes ?
- Supposons que la base de données contienne plusieurs exemplaires de deux livres distincts du même auteur, Michel Tournier, chez le même éditeur, Poche, et portant le même titre : Vendredi ou la vie sauvage. Le bibliothécaire s'aperçoit que l'un des deux livres n'est pas édité par la maison d'édition Poche, mais par Broché. Comment corriger le problème dans la base ?

Questions (4) :

- Supposons que la base de données contienne plusieurs exemplaires de deux livres distincts du même auteur, Michel Tournier, chez le même éditeur, Poche, et portant le même titre : Vendredi ou la vie sauvage. Le bibliothécaire s'aperçoit que l'un des deux livres n'est pas édité par la maison d'édition Poche, mais par Broché. Comment corriger le problème dans la base ?

Enjeux

Illustration sur un cas pratique

Que retenir de cet exemple :

- Redondance des données
- Incohérence en modification
- Anomalie d'insertion
- Anomalie de suppression

Enjeux

Demain

- Gérer des centaines de Tera-octets de données
- Données géographiquement distribuées à l'échelle d'Internet
- Plusieurs dizaines de milliers d'utilisateurs
- Un contexte d'exploitation changeant (débits de communication)
- Voir sur des noeuds volatiles